

Documentación clean code:

1. Nombres Significativos (Meaningful Names):

```
def proc(t, d, u_id=None, l_id=None, dt=None, dt2=None):
    # t: tipo operacion, d: datos, u_id: id usuario, l_id: id libro, dt: fecha, dt2: fecha2
    if t == 1:
        # Registrar usuario
        if d[0] != "" and d[1] != "" and "@" in d[1]:
            if d[2] == "E" or d[2] == "P" or d[2] == "A":
                # E=Estudiante, P=Profesor, A=Administrativo
                nid = len(usuarios_db) + 1
                usuarios_db.append({"id": nid, "n": d[0], "e": d[1], "tipo": d[2], "mult": 0,
"act": True})
                print("Usuario registrado: " + d[0])
                return nid
            else:
                return -1
        else:
            return -1
    elif t == 2:
        # Registrar libro
        if d[0] != "" and d[1] != "" and d[2] > 0:
            nid = len(libros_db) + 1
            libros_db.append({"id": nid, "tit": d[0], "aut": d[1], "ej": d[2], "disp": d[2]})
```

- Variable t confusa
- Variable d confusa
- Variable u_id confusa
- Variable l_id confusa
- Variable dt confusa
- Variable dt2 confusa

2. Funciones Pequeñas y de Única Responsabilidad (Small Functions, Single Responsibility):

```
7 > def proc(t, d, u_id=None, l_id=None, dt=None, dt2=None): ...
152 if __name__ == "__main__":
```

- Nombre de la función no es explicativa ni intuitiva
Estos nombres no tienen propósito ni son fáciles de recordar ni encontrar.
- Función con más de 100 líneas de código lo que hace la función difícil de entender, probar y reutilizar.
- La función es muy extensa y no cumple con el principio de responsabilidad única SOLID.

3. Comentarios: Útiles, No Ruidosos

```
# Version 0.3 - Funciona, no tocar - JP 2023
```

- Faltan comentarios explicativos de la función que defina y esclarezca la lógica empleada
- no demuestra conocimiento del código y no se da mucho contexto para alguien que herede el código.

4. Formato Consistente (Consistent Formatting):

- No se identifica un estilo de nombrado de variables y debido a al desorden se puede caer fácil en errores de indentación lo que es crítico en python

5. No te Repitas (DRY - Don't Repeat Yourself)

```
print("=== Reporte ===")
print("Nombre: " + u["n"])
print("Email: " + u["e"])
if u["tipo"] == "E":
    print("Tipo: Estudiante")
elif u["tipo"] == "P":
    print("Tipo: Profesor")
else:
    print("Tipo: Administrativo")
print("Multas: $" + str(u["mult"]))
```

- no se crean abstracciones (como funciones o clases) para utilizar.

6. Mantenlo Simple y Estúpido (KISS - Keep It Simple, Stupid)

- Mala propuesta al usar condicionales y mal manejo de condicionales, no cumple con el principio KISS ni DRY debido a la redundancia y complejidad innecesaria del código.

```
def proc(t, d, u_id=None, l_id=None, dt=None, dt2=None):
    # t: tipo operacion, d: datos, u_id: id usuario, l_id: id libro, dt: fecha, dt2: fecha2
    if t == 1:
        # Registrar usuario
        if d[0] != "" and d[1] != "" and "@" in d[1]:
            if d[2] == "E" or d[2] == "P" or d[2] == "A":
                # E=Estudiante, P=Profesor, A=Administrativo
                nid = len(usuarios_db) + 1
                usuarios_db.append({"id": nid, "n": d[0], "e": d[1], "tipo": d[2], "mult": 0,
"act": True})
                print("Usuario registrado: " + d[0])
                return nid
            else:
                return -1
        else:
            return -1
```

- Las salidas en **Generación de reporte** saldrían mal debido a el **return** no está situado de forma correcta, por lo que saldría incompleta o solo saldría el elemento que está en la primer posición del vector debido a que el **break**

8. Evitar Números Mágicos y Cadenas Mágicas

```
if u["tipo"] == "E" and c >= 3:
    print("Error: limite de prestamos alcanzado")
    return -1
if u["tipo"] == "P" and c >= 5:
    print("Error: limite de prestamos alcanzado")
    return -1
if u["tipo"] == "A" and c >= 2:
    print("Error: limite de prestamos alcanzado")
    return -1
l = None
for x in libros_db:
    if x["id"] == l_id:
        l = x
        break
if l == None:
    print("Error: libro no existe")
    return -1
if l["disp"] <= 0:
    print("Error: no hay ejemplares disponibles")
    return -1
if u["tipo"] == "E":
    fd = dt + datetime.timedelta(days=7)
elif u["tipo"] == "P":
    fd = dt + datetime.timedelta(days=14)
else:
    fd = dt + datetime.timedelta(days=5)
nid = len(prestamos_db) + 1
prestamos_db.append({"id": nid, "u": u_id, "l": l_id, "fp": dt, "fd_esp": fd, "dev":
None}))

l["disp"] = l["disp"] - 1
print("Prestamo registrado")
return nid

elif t == 4:
    # Devolver libro
    p = None
    for x in prestamos_db:
        if x["id"] == d:
            p = x
            break
    if p == None:
        return -1
    if p["dev"] != None:
        print("Error: ya fue devuelto")
        return -1
    p["dev"] = dt2
    for x in libros_db:
        if x["id"] == p["l"]:
            x["diso"] = x["diso"] + 1
```

- la lógica de validación incluye muchos condicionales anidados y comparaciones genéricas
- Las salidas no se recorren por lo que saldría incompleta o solo saldría el elemento que está en la primer posición del vector debido a que el break no está situado de forma correcta

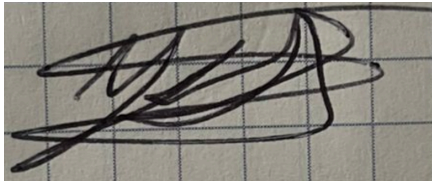
```
elif t == 5:
    # Reporte de usuario
    u = None
    for x in usuarios_db:
        if x["id"] == u_id:
            u = x
            break
    if u == None:
        return -1
```

Fecha: 21 de mayo de 2026

Firma 1: ALVARO JAVIER ARROYO YEPES

Alvaro Javier Arroyo
FIRMA

Firma 2: Yeixon Gembuel

A handwritten signature in black ink on a light-colored grid background. The signature is stylized, featuring a large, sweeping 'Y' shape that loops back and ends with a horizontal stroke.